

Assignment 2 — RISC-V Code Generation

By Mehmed Mulalic - 12415912

1. Overview

This document describes the design and implementation of Assignment 2, which extends the Assignment 1 parser and AST with a full RISC-V assembly. The compiler accepts a restricted subset of C, constructs an Abstract Syntax Tree (AST) during parsing, and then traverses that tree to emit a `program.s` assembly file that can be assembled and run on any RISC-V simulator or hardware target. The implementation adds support for integer arithmetic, relational and logical operators, control-flow constructs (if/else, while, do-while, for), and single-precision floating-point arithmetic with implicit integer-to-float promotion.

2. Project Structure

Changes from Assignment 1 are concentrated in `intermediate.h` and `intermediate.c` with minor changes in the parser to reflect changes in the intermediate file. The scanner remains unchanged and the Makefile contains minor edits such as removing DOT image generation.

File	Description
Makefile	Build files and generates RISC-V assembly as well as file cleanup.
intermediate.h	Struct definitions, constants, extern declarations, and function declarations.
intermediate.c	AST node generation, symbol table, float constant interning, full code generation.
parser.y	Bison grammar. Sets <code>current_type</code> , builds AST nodes via <code>make_*</code> helpers.
scanner.l	Flex tokenizer. Recognizes integer and float literals, keywords, and operators.
test_function.c	A simple C test function to showcase assembly generation.
program.s	Generated RISC-V assembly output.

3. AST Extensions for Code Generation

3.1 NodeData op and NodeKind kind Fields

Binary expression nodes previously stored the operation name as a plain string. A dedicated `NodeData op` enum field was added to `ASTNode`, allowing the code generator to use a switch statement to select the appropriate operation. Additionally, another `NodeKind kind` enum field was included to represent the logical operation of the node.

3.2 Type Propagation

Each node's `int type` field is set during AST construction such that code generation does not require additional searches to infer its type. This field requires the constructed `parser.tab.h` header file which defines each type as a `yymtokentype` enum with integer values.

Identifier nodes inherit their type from the symbol table. Constant nodes inspect the literal string: if it contains a decimal point or exponent character it is tagged `FLOAT`, otherwise `INT`. Importantly, integer literals are always typed as `INT` regardless of the surrounding declaration context — if promotion is needed it is handled at the operation site by `promote_if_needed`. Binary nodes propagate the type upward: if either child is float, the result is float.

```
/* make_binop - type propagation */
node->type = (IS_FLOAT(node_left->type) || IS_FLOAT(node_right->type))
             ? FLOAT : node_left->type;

/* make_constant - literal typing */
if (strchr(name, '.') || strchr(name, 'e') || strchr(name, 'E'))
    ? FLOAT : INT;
```

3.3 Memory Layout

The `ASTNode` data structure includes two additional fields to support code generation — `int offset` and `char *float_label`. The `offset` field represents the memory offset where the node value is stored while the `float_label` dictates the data label to which the value is categorised if the value is a floating number.

4. Symbol Table and Stack Frame Layout

The symbol table is a fixed-size global array of `SymbolStatement` entries (maximum 512). Each entry stores the variable name, its type token, and its frame-pointer-relative offset. Variables are allocated in declaration order, each consuming 4 bytes, with offsets starting at `fp-4` and growing negatively.

The frame is set up in `generate_symbol_code`. The total allocation is rounded up to a 16-byte boundary for ABI alignment and then extended by `TEMP_REG_ALLOC` (160 bytes, 40 words) for the expression stack described in Section 5.

```
int times = (symbol_count + 3) / 4;
int bytes = times * 16 + TEMP_REG_ALLOC;
addi sp, sp, -<bytes>
mv fp, sp
```

5. Expression Stack

Evaluating a binary expression requires preserving the left-hand result while the right-hand sub-expression is computed. A naive implementation using the hardware stack (`addi sp, sp, -4 / sw t0, 0(sp)`) fails for nested expressions: each recursive call moves `sp`, so the subsequent `lw t1, 0(sp)` reads from the wrong address.

The solution uses fp-relative indexed slots. A global counter `expr_depth` tracks nesting level. Before recursing into the right child, the left result is saved to `fp + (expr_depth + 1) * 4` and `expr_depth` is incremented. After the right child returns, the left result is reloaded from the same fixed address and `expr_depth` is decremented. The save and reload instructions must always match, both keyed on `node->left->type`.

```
int is_float_expr = IS_FLOAT(node->type);
int mem = (expr_depth + 1) * 4;

generate_node_code(f, node->left);
if (IS_FLOAT(node->left->type))
    fsw ft0, mem(fp)
else
    sw t0, mem(fp)
expr_depth++;

generate_node_code(f, node->right);
if (IS_FLOAT(node->left->type))
    flw ft1, mem(fp)
else
    lw t1, mem(fp)
expr_depth--;

promote_if_needed(f, node->left->type, node->right->type);
```

6. Floating-Point Support

6.1 Float constant interning

Float literals cannot be loaded using `li` because that instruction only accepts integer immediates. Instead, each unique float literal is registered in a global `FloatConst` table by `intern_float_const` at AST construction time. Before emitting `.text`, `generate_code` writes a `.data` section with one `.float` directive per unique literal. The assembler converts the decimal string to its IEEE 754 bit pattern.

```
.data
.FC0: .float 0.5                # Store the floating value

.text
.globl main

main:
    lui t0, %hi(.FC0)           # Load upper 20 bits of FC0's address
    flw ft0, %lo(.FC0)(t0)      # Include lower 12 bits and load the
full                             address to ft0
```

The `%hi/%lo` are assembler relocations that split a 32-bit address across two instructions, which is necessary because no single RISC-V instruction can encode a full 32-bit immediate.

6.2 Implicit integer-to-float Promotion

When one operand is integer and the other is float, `promote_if_needed` emits a `fcvt.s.w` conversion after both operands are in their respective registers.

7. Limitations

The following are known limitations of the current implementation. They are deliberate simplifications given the assignment scope rather than oversights.

Declarations must precede statements

The symbol table is populated during parsing. Any variable used before its declaration causes lookup to return NULL and the identifier node is silently dropped. All variable declarations must appear at the top of the function body before any statements, matching traditional C89 style.

No char or short support

The allocator always reserves 4 bytes per variable (`current_offset += 4`) and always uses 32-bit load/store instructions (`lw/sw`). Declaring a `char` or `short` variable will assemble without error but use the wrong memory width; the correct instructions would be `lb/sb` and `lh/sh` respectively.

No double support

`IS_FLOAT` checks only for the `FLOAT` token. The `DOUBLE` token exists in the grammar but falls through to the integer code path silently, producing incorrect assembly.

Single function scope only

The back-end targets a single main function. There is no mechanism for function calls, return values, caller-saved registers, or nested scopes. The frame is set up once and never torn down.

No pointers or arrays

The grammar and AST have no representation for address-of, dereference, or indexed access. These constructs would require additional node kinds and addressing modes.

No I/O

Results are only observable by inspecting memory after execution. There is no mechanism to call `printf` or any other library function.

Expression depth limit

The fp-relative expression stack supports a maximum nesting depth of `MAX_EXPR_DEPTH` (40). Expressions deeper than this emit an error to `stderr` and halt code generation for that node. In practice this limit is not reached by realistic input programs.